Plateforme de Partage de Résumés de Cours

INFOH303 – Bases de Données · Partie 2

Gharbi Marwa El Haichami Ilyas Benjelloun Mohamed Chaabane Youssef

Université Libre de Bruxelles · Groupe AB

2025–2026

Plan de la présentation

Application & Base de données

- Architecture & choix techniques
- Structure DB : tables, contraintes, vues
- Triggers PostgreSQL (règles métier + contraintes temporelles)
- Index de performance
- Mécaniques : points & transactions
- Mécaniques : système de niveaux
- Normalisation (1NF $\rightarrow$ BCNF)
- Fonctionnalités & chargement des données

Requêtes analytiques

- Q1 – Top 10 utilisateurs par points
- Q2 – Utilisateurs actifs ($\geq$ 3 cours)
- Q3 – Cours avec le plus de résumés
- Q4 – Résumés les mieux notés par cours
- Q5 – Utilisateurs sans résumé
- Q6 – Objet cosmétique le plus acheté
- Q7 – Utilisateurs en déficit de points
- Q8 – Moyenne de résumés par utilisateur

Architecture de l'application

Couche Présentation

- Framework : **Flask 3**
- Templates : **Jinja2**
- UI : Bootstrap 5 (thème sombre)
- Icônes : Font Awesome 6
- Graphiques : Chart.js 4
- Lancement : `python3 run.py`

Couche Logique

- `web/app.py` : routes Flask
- `app/db.py` : connexion DB
- Sessions Flask (authentification)
- API JSON interne (`/api/query/n`)
- `load_data.py` : import données

Couche Données

- **PostgreSQL 14+**
- psycopg2 (connecteur Python)
- Socket Unix (auth peer)
- 8 tables relationnelles
- 8 triggers côté serveur

`Navigateur → Flask (port 5000) → psycopg2 → PostgreSQL (socket Unix)`

Choix techniques

Composant	Choix	Raison
Backend	Flask 3	Léger, simple à déployer
Base de données	PostgreSQL	Triggers, ACID, robustesse
Connecteur	psycopg2	Standard Python/PostgreSQL
UI Framework	Bootstrap 5	Responsive, thème sombre
Graphiques	Chart.js 4	Léger, intégration JS

Décisions d'architecture

- Connexion via **socket Unix** (auth peer, sans mot de passe)
- Page `/setup` automatique si la DB est inaccessible
- `ON CONFLICT DO NOTHING` + contraintes `UNIQUE` $\Rightarrow$ script idempotent
- Triggers DB pour garantir la cohérence métier
- Transactions ACID applicatives pour les opérations multi-tables
- API JSON interne pour alimenter Chart.js

Structure de la base de données – Tables & Contraintes

Tables (8)

Table	Rôle
UTILISATEUR	Comptes, points, niveau
COURS	Catalogue des cours
RESUME	Résumés, visibilité, note
EVALUATION	Notes 1–5 et commentaires
TRANSACTION_POINTS	Historique gains/dépenses
OBJET_COSMETIQUE	Badges, titres, thèmes
POSSEDE	Achats et objets actifs
LEADERBOARD	Snapshots mensuels du classement

Contraintes d'intégrité

- PK partout, FK avec `ON DELETE CASCADE` (y compris sur TRANSACTION_POINTS : NULL violerait le CHECK "exactement 1 non-NULL")
- `CHECK note BETWEEN 1 AND 5`, `visibilite IN ('public','privé')`, `type_cosmetique IN ('badge','titre','theme')`, `montant > 0`
- `UNIQUE (id_util, id_resume)` sur EVALUATION
- `UNIQUE (id_util, type_periode, date_classement)` sur LEADERBOARD ; `UNIQUE (nom)` sur OBJET_COSMETIQUE
- `CHECK` : exactement 1 parmi `id_eval`, `id_resume`, `id_objet` non NULL dans TRANSACTION_POINTS

Vues

- `v_user_badge` – badge actif de chaque utilisateur
- `v_user_titre` – titre de certification le plus élevé

Design LEADERBOARD – Table vs Vue

Alternative – Vue (non retenue)

- Classement recalculé à chaque requête
- Toujours à jour
- **Pas d'historique** : impossible de connaître le classement du mois précédent

Notre choix – Table avec snapshots

- Snapshot mensuel persisté en base
- `type_periode IN ('mensuel', 'annuel')` : prévu pour plusieurs granularités
- Conserve l'**historique** des classements passés mois par mois

Fonctionnement

- À chaque visite, snapshot du mois courant enregistré dans LEADERBOARD si inexistant
- **Classement temps réel** : requête directe sur `UTILISATEUR ORDER BY nbr_pt DESC`
- **Top 5 du mois précédent** affiché via `LEADERBOARD JOIN UTILISATEUR`
- Deux sources de données coexistent sur la même page

Résultat

La table LEADERBOARD est **réellement utilisée** pour afficher l'historique mensuel dans l'interface, tout en conservant un classement temps réel depuis `UTILISATEUR`.

Triggers PostgreSQL – Règles métier (1/2)

trg_note_moyenne

- Événement : `AFTER INSERT / UPDATE / DELETE` sur EVALUATION
- Action : recalcule `AVG(note)` et met à jour `RESUME.note_moyenne`
- Garantit que la note affichée est toujours synchronisée

trg_auto_evaluation

- Événement : `BEFORE INSERT OR UPDATE` sur EVALUATION
- Action : lève une exception si auteur du résumé = évaluateur
- Règle métier fondamentale, vérifiée côté DB (pas seulement côté UI)

trg_actif_unique

- Événement : `BEFORE INSERT / UPDATE` sur POSSEDE
- Action : désactive l'autre badge actif de l'utilisateur
- Les titres ne sont pas concernés : certifications permanentes

trg_niveau_resume & trg_niveau_note

- Événements : `AFTER INSERT/DELETE` sur RESUME et `AFTER UPDATE OF note_moyenne` sur RESUME
- Même fonction `fn_recalc_niveau()` pour les deux
- Action : recalcule le niveau, attribue les titres par palier

Triggers PostgreSQL – Contraintes temporelles (2/2)

trg_eval_date

- Évén. : `BEFORE INSERT` sur EVALUATION
- `date_eval` $\geq$ `date_publication` du résumé évalué

trg_transaction_date

- Évén. : `BEFORE INSERT` sur TRANSACTION_POINTS
- `date_transaction` $\geq$ `date_inscription` de l'utilisateur

trg_achat_date

- Évén. : `BEFORE INSERT` sur POSSEDE
- `date_achat` $\geq$ `date_inscription` de l'utilisateur

Pourquoi des triggers et non des `CHECK` ?

Ces contraintes référencent une **autre table** (`UTILISATEUR`, `RESUME`).

PostgreSQL interdit les sous-requêtes dans les contraintes `CHECK`.

$\Rightarrow$ Trigger `BEFORE INSERT` obligatoire pour toute contrainte inter-tables.

Index de performance

Pourquoi des index ? Les PK sont automatiquement indexées par PostgreSQL. Ces 8 index couvrent les FK utilisées en `WHERE`/`JOIN` pour éviter les sequential scans.

Colonne indexée	Utilité
`RESUME(id_utilisateur)`	mes résumés, Q2
`RESUME(code_cours)`	Q3, Q4
`EVALUATION(id_resume)`	trg_note_moyenne
`EVALUATION(id_utilisateur)`	anti auto-éval

Colonne indexée	Utilité
`TRANSACTION(id_utilisateur)`	historique, Q7
`TRANSACTION(date_transaction)`	tri chronologique
`POSSEDE(id_utilisateur)`	boutique, Q6
`LEADERBOARD(id_utilisateur)`	classement

Impact direct sur les requêtes analytiques

Q2, Q3, Q4, Q6, Q7 bénéficient directement de ces index : les `GROUP BY` et jointures évitent les scans complets de table.

Mécaniques métier – Système de points

Règles d'attribution

- **+10 pts** – publication d'un résumé
- **+5 pts** – évaluation donnée (évaluateur)
- **+5 pts** – évaluation reçue (auteur du résumé)
- **−prix** – achat d'un objet cosmétique

Table TRANSACTION_POINTS

- Chaque mouvement est enregistré avec son origine
- Contrainte `CHECK` : exactement 1 parmi `id_eval`, `id_resume`, `id_objet` est non-NULL
- Permet de reconstituer l'historique complet par type

Chaîne de triggers lors d'une évaluation

INSERT INTO EVALUATION

↓ *trg_note_moyenne*

UPDATE RESUME.note_moyenne

↓ *trg_niveau_note*

UPDATE UTILISATEUR.niveau

↓ *si montée de palier*

INSERT INTO POSSEDE
(titre de certification automatique)

Une seule insertion déclenche la cascade note_moyenne → niveau → titre.
La mise à jour de `nbr_pt` est gérée côté application pour un contrôle transactionnel précis.

Mécaniques métier – Stockage des points (1/2)

Phase 1 : chargement initial (`load_data.py`)

- `nbr_pt` **copié tel quel** depuis `<points>` du XML
- Transactions **reconstruites** : +10/résumé, +5/éval., −prix/achat
- Solde XML $\neq \sum$ transactions *(approximation — historique réel inconnu)*

Phase 2 : utilisation de l'app (dynamique)

- `nbr_pt` et transaction mis à jour **simultanément** dans la même transaction SQL
- `nbr_pt` = solde réel à tout instant
- $\sum$ transactions = historique complet et cohérent
- Si l'une échoue ⇒ rollback complet (ACID)

INSERT INTO RESUME
→ **INSERT INTO TRANSACTION_POINTS** `(montant=10, type='gain')`
→ **UPDATE UTILISATEUR SET nbr_pt = nbr_pt + 10**
tout dans un seul `db.transaction()`

Mécaniques métier – Stockage des points (2/2)

Pourquoi stocker `nbr_pt` si on a déjà `TRANSACTION_POINTS` ?

Alternative "pure" (non retenue)

- Ne pas stocker `nbr_pt`
- Calculer à la volée : `SELECT SUM(montant) FROM TRANSACTION_POINTS WHERE id_utilisateur = X`
- **Problème** : agrégation coûteuse exécutée sur chaque page (*dashboard*, *classement*, *boutique*, *shop*...)

Notre choix – dénormalisation assumée

- `nbr_pt` stocké directement dans `UTILISATEUR`
- Lecture instantanée : `SELECT nbr_pt FROM UTILISATEUR`
- Cohérence garantie par `db.transaction()` (ACID)

*`TRANSACTION_POINTS` sert à **afficher l'historique** (page "Mes Points") et aux **requêtes analytiques** (ex. Q7 : utilisateurs en déficit) — `nbr_pt` est la **source de vérité** du solde.*

Mécaniques métier – Système de niveaux

Formule du score

$$\text{score} = \underbrace{n_{\text{résumés}}}_{\text{quantité (tous)}} \times \underbrace{\overline{\text{note}}}_{\text{qualité (notés)}}$$

Récompense quantité *et* qualité. COUNT porte sur tous les résumés ; AVG ignore les `NULL` (résumés non encore notés).

Déclencheurs

- `trg_niveau_resume` : après ajout/suppression d'un résumé
- `trg_niveau_note` : après recalcul de la note moyenne
- Les deux appellent la même fonction `fn_recalc_niveau()`

Paliers et titres automatiques

Score	Niveau	Titre de certification
0	1	–
≥ 1	2	Étudiant modèle
≥ 3	3	Mentor
≥ 6	4	Top Contributeur
≥ 10	5	Maître du savoir
≥ 15	6	Professeur
≥ 22	7	VIP
≥ 32	8	Élite
≥ 45	9	Légende du campus

Titres : certifications permanentes

Non achetables, insérés dans `POSSEDE` par trigger. `est_actif` ne s'applique qu'aux badges. Titre affiché via la vue `v_user_titre`.

Analyse de normalisation – Dépendances fonctionnelles

Dépendances fonctionnelles par table

Table	Dépendances fonctionnelles
UTILISATEUR	*id* → nom, email, date, nbr_pt, niveau *nom* → . . ., *email* → . . . (clés candidates)
COURS	*code_cours* → nom, faculte, annee
RESUME	*id_resume* → titre, date, visibilite, note_moy, id_util, code_cours
EVALUATION	*id_eval* → note, commentaire, date, id_util, id_resume (*id_util*, *id_resume*) → *id_eval*, . . .
OBJET_COSM.	*id_objet* → nom, desc, prix, type, niv_requis *nom* → . . . (clé candidate)
POSSEDE	(*id_util*, *id_objet*) → date, est_actif
TRANSACTION	*id_tx* → montant, type, date, id_util, id_resume, id_objet, id_eval
LEADERBOARD	*id_cls* → rang, nbr_pt, periode, date, id_util (*id_util*, *periode*, *date*) → . . . (clé cand.)

Clés candidates identifiées

- **UTILISATEUR** : {id_utilisateur}, {nom_utilisateur}, {email}
- **COURS** : {code_cours}
- **RESUME** : {id_resume}
- **EVALUATION** : {id_eval} et {id_utilisateur, id_resume}
- **OBJET_COSMETIQUE** : {id_objet}, {nom}
- **POSSEDE** : {id_utilisateur, id_objet}
- **TRANSACTION_POINTS** : {id_transaction}
- **LEADERBOARD** : {id_classement}, {id_util, type_periode, date_classement}

Vérification 1NF

- Tous les attributs sont atomiques ✓
- Chaque table possède une clé primaire ✓
- Aucune valeur multivaluée ou répétée ✓

Analyse de normalisation – 2NF, 3NF et BCNF

2NF – Pas de dépendances partielles

- Toutes les tables ont une PK mono-attribut *sauf* POSSEDE et EVALUATION
- **POSSEDE**(id_util, id_objet) $\rightarrow$ $date, est_actif$: `date` et `est_actif` dépendent de *toute* la PK composite ✓
- **EVALUATION**(id_util, id_resume) $\rightarrow$ $note, commentaire$:
  Les attributs dépendent de *toute* la PK ✓
- **Conclusion : schéma en 2NF** ✓

3NF – Pas de dépendances transitives

- Aucune dépendance transitive non-triviale entre attributs non-clés ✓
- **EVALUATION** : deux clés candidates $\Rightarrow$ chaque DF a un déterminant superclé ✓
- **Conclusion : schéma en 3NF** ✓

BCNF – Tout déterminant est superclé

Table	3NF	BCNF
UTILISATEUR	✓	✓
COURS	✓	✓
RESUME	✓	✓
EVALUATION	✓	✓
OBJET_COSMETIQUE	✓	✓
POSSEDE	✓	✓
TRANSACTION_POINTS	✓	✓
LEADERBOARD	✓	✓

Dénormalisations volontaires assumées

Toutes les tables sont en BCNF. Cependant, `UTILISATEUR.nbr_pt` et `RESUME.note_moyenne` sont dérivables depuis d'autres tables : c'est une **redondance inter-tables** (distincte d'une violation BCNF, qui ne porte que sur les DF internes à la relation).
Choix délibéré : stockés pour la performance (lecture directe sans agrégation) et maintenus cohérents par trigger (`note_moyenne`) et par `db.transaction()` (`nbr_pt`).

Fonctionnalités implémentées

Utilisateurs & Cours

- Inscription & connexion
- Tableau de bord (points, rang)
- Catalogue de 47 cours (+ 2 stubs auto-créés)
- Recherche et ajout de cours

Résumés & Évaluations

- Publication (+10 pts)
- Visibilité public / privé
- Page détail avec commentaires
- Évaluation 1–5 étoiles (+5 pts)
- Filtre mes résumés / tous
- Suppression de ses résumés

Points & Boutique

- Solde et historique transactions
- 45 objets cosmétiques
- Achat avec vérif. du solde
- Activation badge (max 1 actif)
- Titres : récompenses de niveau toujours visibles
- Classement avec podium top 3

Dashboard analytique

8 indicateurs clés en temps réel : top 10 (graphique barres), cours le plus actif, objet populaire, utilisateurs sans résumé, résumés les mieux notés, déficit de points, moyenne de résumés/utilisateur.

Chargement des données fournies

Fichiers parsés par `load_data.py`

Fichier	Format	Résultat
`cours.csv`	CSV	47 cours
`recompenses.xml`	XML	45 objets cosmétiques
`utilisateurs`	XML	50 utilisateurs, 89 résumés
`commentaires.json`	JSON	87 évaluations
Transactions générées		391

Robustesse du script

- `ON CONFLICT DO NOTHING` : relançable sans doublons
- Cours créés à la volée si absents du CSV
- Type *cosmetique* → *theme* (contrainte CHECK)
- Champs vides traités avec valeurs par défaut
- +10 pts par résumé publié
- +5 pts à l'évaluateur et à l'auteur
- Note moyenne recalculée via trigger

Limitations assumées

- `nbr_pt` copié du XML, tx reconstruites ⇒ solde $\neq \sum$ tx (slide suivant)
- `<niveau>` XML ignoré : attribut dérivé, recalculé par le trigger (sinon des paliers sautés → titres manquants)

Données fournies – Solde vs historique reconstruit

Pourquoi ne pas recalculer `nbr_pt` depuis les événements ?

Alternative – recalculer depuis les events

- $\texttt{nbr_pt} = \sum \text{gains}_{reconstruits} - \sum \text{dépenses}_{reconstruites}$
- Top 4 utilisateurs après recalcul :

user	XML	recalculé
emma_bernard	3200	−8745
yoann_bell	2600	−6540
sarah_evans	2900	−8050
isabelle_lambert	2750	−6550

$\Rightarrow$ tout le top clampé à 0 par le CHECK ≥ 0. Classement inutilisable.

Notre choix – garder le <points> XML

- Préserve la curation des données fournies
- Classement riche et différencié
- Q7 matérialise l'écart **visuellement** (sonde d'audit)

Pourquoi l'écart ?

Les <achats> listés (jusqu'à 8800 pts) dépassent largement les <resumes> $\times$ +10 (55 pts). Le fichier n'expose pas l'historique complet des gains passés (résumés supprimés, bonus...). On n'a pas pu inventer ces transactions fantômes sans violer le CHECK "exactement 1 lien non-NULL" sur TRANSACTION_POINTS.

Données fournies – Anomalies structurelles détectées

3 entrées suspectes dans `utilisateurs.xml`

Cas	Code reçu	Code attendu	Notre traitement
Balise vide	`<cours></cours>`	–	**Rejet** (PK vide invalide)
Typo	`INOH307`	`INFOH307`	**Stub** `faculte='Inconnu'`
Typo	`PHYS20`	`PHYS201`	**Stub** `faculte='Inconnu'`

Pourquoi pas corriger automatiquement ?

- `INOH307` → `INFOH307` : évident
- `PHYS20` → `PHYS201` : déduit du titre du résumé ("Optique"), pas du code
- Corriger à notre niveau = **modifier les données fournies** sans autorisation

Notre choix – les rendre visibles

- PK vide : **filtrée** (violation conceptuelle)
- Typos : insérés en stub avec `faculte='Inconnu'` ⇒ visibles dans `/courses`
- Le SGBD aurait normalement rejeté la FK : on garde l'esprit en signalant la dirty data

Q1 – Top 10 utilisateurs par points

```sql
SELECT nom_utilisateur, nbr_pt
FROM   UTILISATEUR
ORDER BY nbr_pt DESC,
         date_inscription ASC
LIMIT  10;
```

Retourne les 10 utilisateurs ayant accumulé le plus de points, triés par ordre décroissant.

Algèbre Relationnelle × **Non exprimable**

`ORDER BY` et `LIMIT` ne font pas partie de l'AR de Codd. L'AR est un langage ensembliste : elle ne définit pas d'ordre sur les tuples ni de restriction de cardinalité du résultat.

Calcul des Tuples × **Non exprimable**

Le calcul tuple du 1er ordre ne dispose d'aucun mécanisme pour ordonner ou limiter le nombre de tuples retournés. Ces opérations sont procédurales, non déclaratives.

Q2 – Utilisateurs avec résumés dans ≥ 3 cours

```
SELECT u.nom_utilisateur,
  COUNT(DISTINCT r.code_cours) AS
      nb_cours,
  COUNT(r.id_resume) AS nb_resumes
FROM   UTILISATEUR u, RESUME r
WHERE  r.id_utilisateur = u.
    id_utilisateur
GROUP BY u.id_utilisateur, u.
    nom_utilisateur
HAVING COUNT(DISTINCT r.code_cours) >=
    3
ORDER BY nb_cours DESC, u.
    nom_utilisateur;
```

Algèbre Relationnelle × **Non exprimable**

`COUNT DISTINCT` et `GROUP BY` sont absents de l'AR de Codd. L'AR est un langage ensembliste : elle ne dispose d'aucun opérateur d'agrégation.

Calcul des Tuples × **Non exprimable**

`COUNT DISTINCT` nécessite une agrégation absente du calcul tuple du 1er ordre. Une approximation avec 3 existentiels imbriqués est possible mais ne généralise pas pour N quelconque.

Q3 – Cours ayant le plus de résumés publiés

```
SELECT c.code_cours, c.nom_cours,
  COUNT(r.id_resume) AS total_resumes
FROM   COURS c, RESUME r
WHERE  c.code_cours = r.code_cours
GROUP BY c.code_cours, c.nom_cours
ORDER BY total_resumes DESC
LIMIT 1;
```

Algèbre Relationnelle × **Non exprimable**

`COUNT` et `GROUP BY` sont absents de l'AR de Codd. Le nombre de résumés doit être calculé, non stocké. Trouver le MAX d'un agrégat dépasse les capacités de l'AR standard.

Calcul des Tuples × **Non exprimable**

La logique du 1er ordre peut vérifier l'existence de tuples, mais ne peut pas compter leur nombre ni déterminer un maximum sur un agrégat calculé.

Q4 – Résumés les mieux notés par cours

```sql
SELECT r.code_cours, r.titre,
       r.note_moyenne
FROM   RESUME r
WHERE  r.note_moyenne IS NOT NULL
  AND  r.note_moyenne = (
    SELECT MAX(r2.note_moyenne)
    FROM   RESUME r2
    WHERE  r2.code_cours = r.
        code_cours
)
ORDER BY r.code_cours;
```

Note : `note_moyenne` est stocké dans RESUME (mis à jour automatiquement par trigger).

Algèbre Relationnelle pure ✓ Exprimable

Technique des *perdants* (sans γ) :

$X \leftarrow \text{RESUME}$, $Y \leftarrow \text{RESUME}$

$\text{Perdants} = \pi_{X.id}(\sigma_{Y.note_moy > X.note_moy \wedge Y.cours = X.cours}(X \times Y))$

$\text{Résultat} = (\pi_{id}(\text{RESUME}) - \text{Perdants}) \bowtie \text{RESUME}$

Calcul des Tuples ✓ Exprimable

$\{r \mid \text{RESUME}(r) \wedge$

$\neg\exists r_2(\text{RESUME}(r_2) \wedge$

$r_2.cours = r.cours \wedge$

$r_2.note_moy > r.note_moy)\}$

Q5 – Utilisateurs n'ayant jamais publié de résumé

```sql
SELECT nom_utilisateur, email,
       date_inscription
FROM   UTILISATEUR
WHERE  id_utilisateur NOT IN (
    SELECT id_utilisateur FROM RESUME
)
ORDER BY nom_utilisateur;
```

Algèbre Relationnelle pure ✓ **Exprimable**

$R_1 = \pi_{id}(\text{RESUME})$

$R_2 = \pi_{id}(\text{UTILISATEUR}) - R_1$

Résultat $= \pi_{nom,email,date}(\text{UTILISATEUR} \bowtie R_2)$

Utilise la **différence ensembliste** $(-)$,

opérateur fondamental de l'AR de Codd.

Calcul des Tuples ✓ **Exprimable**

$\{u \mid \text{UTILISATEUR}(u) \wedge$

$\neg\exists r(\text{RESUME}(r) \wedge$

$r.id_utilisateur = u.id_utilisateur)\}$

La négation existentielle $(\neg\exists)$ exprime

directement l'absence de résumé.

Q6 – Objet cosmétique le plus acheté

```sql
SELECT oc.nom, oc.type_cosmetique,
       oc.prix_points,
       COUNT(p.id_utilisateur) AS
           nb_achats
FROM   OBJET_COSMETIQUE oc, POSSEDE p
WHERE  p.id_objet = oc.id_objet
  AND  oc.type_cosmetique != 'titre'
GROUP BY oc.id_objet, oc.nom,
         oc.type_cosmetique, oc.
             prix_points
ORDER BY nb_achats DESC
LIMIT 1;
```

Algèbre Relationnelle × **Non exprimable**

`COUNT` et `GROUP BY` sont absents de l'AR de Codd. Trouver le maximum d'un comptage nécessite deux niveaux d'agrégation, impossibles en AR standard.

Calcul des Tuples × **Non exprimable**

Requiert $MAX(COUNT(\ldots))$, soit deux niveaux d'agrégation imbriqués. Le CT du 1er ordre ne dispose d'aucun opérateur de cardinalité.

Q7 – Utilisateurs en déficit de points

```
SELECT u.nom_utilisateur, u.nbr_pt,
  SUM(CASE WHEN t.type_transaction != '
      gain'
      THEN t.montant ELSE 0 END) AS
          depense,
  SUM(CASE WHEN t.type_transaction = '
      gain'
      THEN t.montant ELSE 0 END) AS gain
FROM UTILISATEUR u, TRANSACTION_POINTS t
WHERE t.id_utilisateur = u.id_utilisateur
GROUP BY u.id_utilisateur,
         u.nom_utilisateur, u.nbr_pt
HAVING SUM(CASE WHEN t.type_transaction
    != 'gain'
          THEN t.montant ELSE 0 END)
     > (u.nbr_pt + SUM(CASE WHEN
          t.type_transaction = 'gain'
          THEN t.montant ELSE 0 END));
```

Note : PostgreSQL n'autorise pas les alias du `SELECT` dans `HAVING` — les expressions `CASE` sont réécrites

Algèbre Relationnelle × **Non exprimable**

`SUM` conditionnel nécessite γ. La comparaison HAVING porte sur deux agrégats distincts.

Calcul des Tuples × **Non exprimable**

Pas de sommes accumulées en CT 1er ordre. $\Sigma(\text{dép.}) > \text{solde} + \Sigma(\text{gain})$ non exprimable avec $\exists/\forall$.

Sémantique **sonde d'audit**

Sur données cohérentes (`db.transaction()` + CHECK ≥ 0), la condition est **toujours fausse**. Elle détecte les écarts `nbr_pt` vs historique : utile car `load_data.py` importe le solde XML sans transactions correspondantes.

Q8 – Nombre moyen de résumés par utilisateur

```sql
SELECT
  ROUND(
    (SELECT COUNT(*) FROM RESUME)::
        numeric -- cast: evite
        division entiere
    / NULLIF((SELECT COUNT(*) FROM
        UTILISATEUR), 0), -- evite
        div/0
  2) AS moyenne;
```

Division du nombre total de résumés par le nombre total d'utilisateurs. `NULLIF` évite la division par zéro.

Note : bien que l'interface garantisse toujours au moins un utilisateur connecté, la contrainte est maintenue par **défense en profondeur** : la base de données reste cohérente indépendamment de l'application.

Algèbre Relationnelle × **Non exprimable**

Calculer une moyenne nécessite `COUNT` et une division entre deux agrégats. Ces opérations sont absentes de l'AR de Codd.

Calcul des Tuples × **Non exprimable**

Calculer une moyenne nécessite COUNT, absent du CT du 1er ordre. La division entre deux cardinaux calculés dynamiquement dépasse les capacités de la logique du 1er ordre.

Conclusion

Ce qui a été réalisé

- Application web Flask complète, déployable localement
- Interface professionnelle avec dashboard analytique
- PostgreSQL : 8 tables, triggers, contraintes d'intégrité
- Script de chargement idempotent (CSV, XML, JSON)
- 8 requêtes analytiques avec visualisations Chart.js
- Système de points, boutique et classement

Bilan sur les formalismes

Requête	AR	CT
Q1 – Top 10	×	×
Q2 – $\geq$ 3 cours	×	×
Q3 – Plus de résumés	×	×
Q4 – Mieux notés	✓	✓
Q5 – Sans résumé	✓	✓
Q6 – Objet pop.	×	×
Q7 – Déficit	×	×
Q8 – Moyenne	×	×